

(SKILL Content Delivery Concepts)

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

```
eesha@DESKTOP-6R05L78:~$ ./myshell
myshell> nano ini.c
myshell> gcc ini.c -o in
myshell> ./in
Linux OSSP project is ready.
GCC and Makefile can be used to build this program.
myshell> |
```

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

2. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
myshell> nano intloop.c
myshell> gcc intloop.c -o l
myshell> ./l
myshell>
You entered:
myshell> exit
Shell exited.
myshell> |
```

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

3. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

```
myshell> gcc recfunc.c -o rf
myshell> ./rf
Enter commands (type history to display, exit to stop).
> history
> exit
myshell> |
```

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

```
myshell> nano deli.c
myshell> gcc deli.c -o del
myshell> ./del
Enter command: ls
Token: ls
myshell> |
```

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
myshell> nano singquo.c
myshell> gcc singquo.c -o sq
myshell> ./sq
Single quoted string: 'hello $USER world'
Content inside single quotes is treated literally.
myshell> |
```

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
myshell> nano procescp.c
myshell> gcc procescp.c -o ps
myshell> ./ps
Parsed: hello world
myshell> |
```

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

7. Process Synchronization and Child Process Monitoring Using waitpid().

```
myshell> nano processync.c
myshell> gcc processync.c -o pc
myshell> ./pc
Parent waiting using waitpid()...
Child finished.
Child has terminated.
myshell> |
```

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

8. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

```
myshell> nano varref.c
myshell> gcc varref.c -o vr
myshell> ./vr
Enter variable name: 68
Variable is undefined.
myshell> |
```

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

```
myshell> nano chnagedir.c
myshell> gcc chnagedir.c -o cd
myshell> ./cd
Enter directory: OSLab
Current directory: /home/eesha/OSLab
myshell> |
```

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

```
Current directory: /home/eesha
Type 'exit' in a shell to process an exit request.
myshell> exit
Shell exited.
eesha@DESKTOP-6R05L78:~$ ./myshell
```

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

```
myshell> nano storecom.c
myshell> gcc storecom.c -o sc
myshell> ./sc
Command 1: ls
Command 2: date
Command 3: history
Command 4: ^C
eesha@DESKTOP-6R05L78:~$ |
```

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

```
myshell> nano red.c
myshell> gcc red.c -o r
myshell> ./r
Parent received: Hello from child process via pipe!
myshell> |
```

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

13. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

```
myshell> nano createpipes.c
myshell> nano inpred.c
myshell> gcc inpred.c -o ir
myshell> ./ir
input.txt: No such file or directory
myshell> |
```

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

14. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

```
myshell> nano parse0.c
myshell> gcc parse0.c -o po
myshell> ./po
myshell> ./ir
myshell> ./po
myshell> nano parse0.c
myshell> ./po
```

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

15. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

```
myshell> nano parsered.c
myshell> gcc parsered.c -o pd
myshell> ./pd
```

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

```
myshell> nano bg.c
myshell> gcc bg.c -o b
myshell> ./b
Background job started. PID=916
Parent returns to prompt immediately.
Background job completed.
myshell> |
```

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

```
myshell> nano activej.c
myshell> gcc activej.c -o aj
myshell> ./aj
[1] PID=926 Status=Running
[1] PID=926 Status=Completed
myshell> |
```

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

18. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

```
[1] PID=926 Status=Completed
myshell> nano sigdel.c
myshell> gcc sigdel.c -o sd
myshell> ./sd
PID: 988
Send SIGUSR1 using: kill -USR1 PID
^C
eesha@DESKTOP-6R05L78:~$ |
```

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

19. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

```
myshell> nano reghand.c
myshell> gcc reghand.c -o rh
myshell> ./rh
PID: 1012
^C
Signal 2 received. Program is stable.
```

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

20. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

```
eesh@DESKTOP-6R05L78:~$ ./myshell
myshell> nano suspproc.c
myshell> gcc suspproc.c -o sp
myshell> ./sp
PID: 1036
Press Ctrl+Z to suspend.
Running...
Running...
Running...
^Z
SIGTSTP received. Process will stop.

[1]+  Stopped                  ./myshell
eesh@DESKTOP-6R05L78:~$ |
```

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

21. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
eesh@DESKTOP-6R05L78:~$ ./myshell
myshell> nano conmodu.c
myshell> gcc conmodu.c -o cm
myshell> ./cm
Module 1 result: 30
End-to-end module test completed.
myshell> |
```

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

22. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
myshell> nano valgrind.c
myshell> gcc valgrind.c -o v
myshell> ./v
Memory allocated successfully.
Memory released successfully.
myshell> |
```

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

23. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

```
myshell> nano lap.c
myshell> gcc lap.c -o lp
myshell> ./lp
Pipeline job 1, PID=1078
Pipeline job 2, PID=1079
Pipeline job 3, PID=1080
All jobs completed.
CPU time: 0.002 seconds
myshell> |
```

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

24. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

```
myshell> nano org.c
myshell> gcc org.c -o o
myshell> ./o
Project structure check
src/      - source files
include/  - header files
build/    - compiled files
README.md - project documentation
Makefile  - build instructions
myshell> |
```

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

```
myshell> gcc dems.c -o d
myshell> ./d
OSSP Demo
PID: 1103
Feature 1: Process execution
Feature 2: Pipelines
Feature 3: Signal handling
Sending demonstration signal...
Demo signal received: 10
Final project review completed.
myshell> |
```